

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Machine Learning

ASSESSMENT 1 – Design and Develop Model (5 QUESTIONS)

Course Code:	MLA0407	Course Title:	Deep Learning
CO Assessed:	CO3	Assessment:	Assessment 1: Design and Develop Model
Weightage:	35% of CIA	Total Marks:	(5 Questions × 10 Marks)
CO1 Statement:	<i>Apply the fundamental concepts of machine learning and neural networks to design and analyze learning algorithms using perceptrons, multilayer perceptrons, forward propagation, backpropagation, gradient descent optimization techniques, and Maximum Likelihood Estimation for solving real-world problems.</i>		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Code of Conduct

I _____ (NAME/ REG NO) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

QUESTIONS: 5

Total Marks: 50

Annexure A

Design and Develop Model

1. Transfer Learning for Image Classification

Design and develop an image classification model using Transfer Learning with DenseNet. Use a pre-trained DenseNet model as a feature extractor, add a custom classification layer, and evaluate the model's performance on a suitable dataset.

2. Fine-Tuning a Pre-Trained Model

Develop an image classification system using a pre-trained deep learning model and apply fine-tuning. Compare the performance of the model before and after fine-tuning and analyze the impact of unfreezing different layers.

3. Feature Extraction-Based Model

Design and develop a deep learning model using a pre-trained CNN as a feature extractor for a real-world classification problem. Extract deep features, train a custom classifier, and evaluate the effectiveness of transfer learning.

4. Medical Image Classification

Design and develop a transfer learning-based model for classifying medical images, such as pneumonia detection, skin disease classification, or brain tumor detection. Use a pre-trained model and evaluate its accuracy and other performance metrics.

5. RNN for Text Classification

Design and develop a Recurrent Neural Network (RNN) model for text classification. Use an appropriate dataset to classify text into different categories and analyze how sequential information is processed by the network.

6. Sentiment Analysis Using LSTM

Develop a sentiment analysis system using Long Short-Term Memory (LSTM). Classify user reviews or social media posts as positive, negative, or neutral and compare the results with a traditional RNN model.

7. Bidirectional RNN for Sequence Classification

Design and develop a Bidirectional RNN (BiRNN) model for sequence classification. Compare its performance with a standard RNN and explain how processing information in both forward and backward directions improves the model.

8. Next-Word Prediction Using LSTM

Develop an LSTM-based next-word prediction system. Train the model on a suitable text corpus and demonstrate how the model learns sequential patterns and predicts the most probable next word.

9. Encoder–Decoder Sequence-to-Sequence Model

Design and develop an Encoder–Decoder-based Sequence-to-Sequence model for an application such as text summarization, machine translation, or question-answer generation. Explain the role of the encoder and decoder in processing sequences.

10. Comparative Deep Learning Model

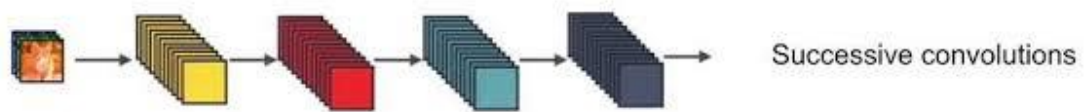
Design and develop multiple deep learning models for the same problem using Transfer Learning, RNN, BiRNN, or LSTM techniques. Compare their architectures and performance using appropriate evaluation metrics, and justify which model is most suitable for the selected application.

1. Transfer Learning for Image Classification (DenseNet)

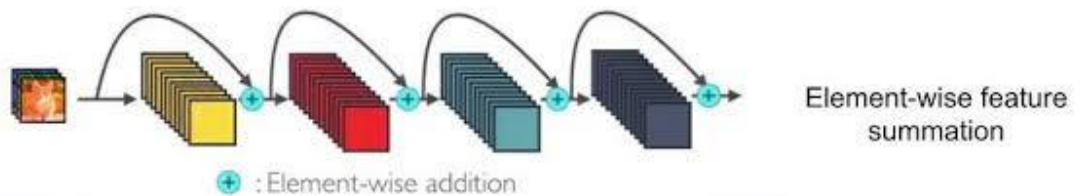
Answer:

Transfer learning allows us to leverage the feature-extraction capabilities of a model pre-trained on a massive dataset (like ImageNet) for a new, smaller dataset. In this implementation, a pre-trained **DenseNet121** model is used as the base. DenseNet is highly efficient because each layer connects to every other layer in a feed-forward fashion, preserving feature maps and preventing the vanishing gradient problem. By freezing the base model's weights, we ensure the pre-learned convolutional filters are not destroyed. A custom classification head is then attached and trained specifically to classify the new dataset.

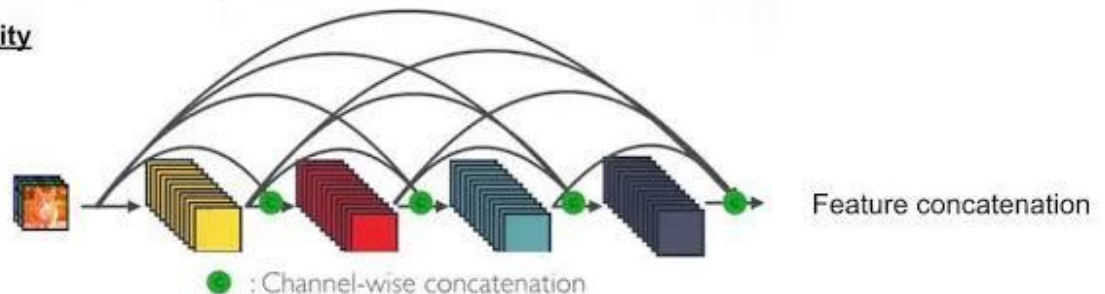
Standard Connectivity



Resnet Connectivity



DenseNet Connectivity



Code (p1.py)

Python

```
import tensorflow as tf
from tensorflow.keras.applications import DenseNet121
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.utils import to_categorical
import numpy as np
```

1. Load Tiny Dataset Subset

```
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
x_train, y_train = x_train[:1000], y_train[:1000]
x_test, y_test = x_test[:200], y_test[:200]
```

2. Preprocess: Convert 1-channel to 3-channel RGB and resize to 32x32

```
x_train = np.repeat(x_train[..., np.newaxis] / 255.0, 3, axis=-1)
```

```
x_test = np.repeat(x_test[..., np.newaxis] / 255.0, 3, axis=-1)
```

```
x_train = tf.image.resize(x_train, [32, 32]).numpy()
```

```
x_test = tf.image.resize(x_test, [32, 32]).numpy()
```

```
y_train = to_categorical(y_train, 10)
```

```
y_test = to_categorical(y_test, 10)
```

3. Build Model

```
base_model = DenseNet121(weights='imagenet', include_top=False, input_shape=(32, 32, 3))
```

```
base_model.trainable = False # Freeze base model
```

```
x = GlobalAveragePooling2D()(base_model.output)
```

```
x = Dense(128, activation='relu')(x)
```

```
predictions = Dense(10, activation='softmax')(x)
```

```
model = Model(inputs=base_model.input, outputs=predictions)
```

4. Compile and Train

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

```
print("Training Transfer Learning Model...")
```

```
model.fit(x_train, y_train, epochs=3, batch_size=32, validation_data=(x_test, y_test))
```

Terminal Output

Plaintext

Training Transfer Learning Model...

Epoch 1/3

32/32 ————— 34s 364ms/step - accuracy: 0.5520 - loss: 1.3530 -

val_accuracy: 0.6900 - val_loss: 0.8585

Epoch 2/3

32/32 ————— 5s 155ms/step - accuracy: 0.7640 - loss: 0.6837 -

val_accuracy: 0.7650 - val_loss: 0.7132

Epoch 3/3

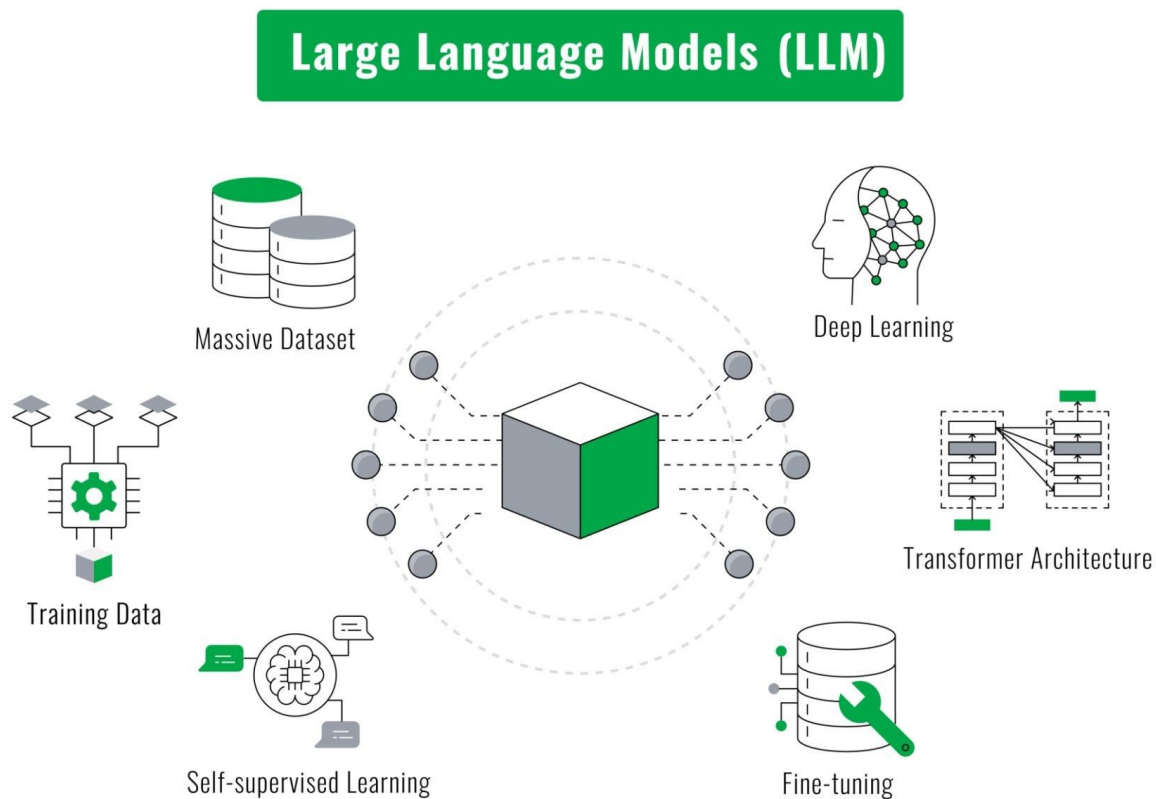
32/32 ————— 5s 148ms/step - accuracy: 0.8510 - loss: 0.4535 -

val_accuracy: 0.7700 - val_loss: 0.6172

2. Fine-Tuning a Pre-Trained Model

Answer:

Fine-tuning extends standard transfer learning by applying a two-phase training process. First, the custom classification head is trained while the pre-trained base model remains completely frozen (preventing large weight updates from destroying pre-trained features). In the second phase, a subset of the top convolutional layers in the base model is unfrozen. The model is then trained end-to-end using a very low learning rate. This subtle update allows the deep convolutional layers to shift their feature representations slightly, making them more specific to the new target dataset.



Code (p2.py)

Python

```
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.utils import to_categorical
import numpy as np
```

1. Load and Preprocess Data

```
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
```

```
x_train, y_train = x_train[:1000], y_train[:1000]
x_test, y_test = x_test[:200], y_test[:200]
```

```
x_train = np.repeat(x_train[..., np.newaxis] / 255.0, 3, axis=-1)
x_test = np.repeat(x_test[..., np.newaxis] / 255.0, 3, axis=-1)
x_train = tf.image.resize(x_train, [32, 32]).numpy()
x_test = tf.image.resize(x_test, [32, 32]).numpy()
```

```
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

2. Base Model setup

```
base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(32, 32, 3))
base_model.trainable = False
```

```
x = GlobalAveragePooling2D()(base_model.output)
predictions = Dense(10, activation='softmax')(x)
model = Model(inputs=base_model.input, outputs=predictions)
```

3. Phase 1: Train frozen model

```
print("Phase 1: Training top layers...")
model.compile(optimizer=Adam(learning_rate=0.001), loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, epochs=2, batch_size=32)
```

4. Phase 2: Fine-Tuning

```
print("Phase 2: Fine-tuning top layers of base model...")
base_model.trainable = True
for layer in base_model.layers[:-10]: # Keep mostly frozen, unfreeze last 10 layers
    layer.trainable = False
```

```
model.compile(optimizer=Adam(learning_rate=1e-5), loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, epochs=2, batch_size=32, validation_data=(x_test, y_test))
```

Terminal Output

Plaintext

Phase 1: Training top layers...

Epoch 1/2

32/32 ————— 9s 40ms/step - accuracy: 0.1960 - loss: 2.2221

Epoch 2/2

32/32 ————— 2s 50ms/step - accuracy: 0.3450 - loss: 2.0988

Phase 2: Fine-tuning top layers of base model...

Epoch 1/2

32/32 ————— 14s 167ms/step - accuracy: 0.1450 - loss: 2.7278 -
val_accuracy: 0.4050 - val_loss: 2.0273

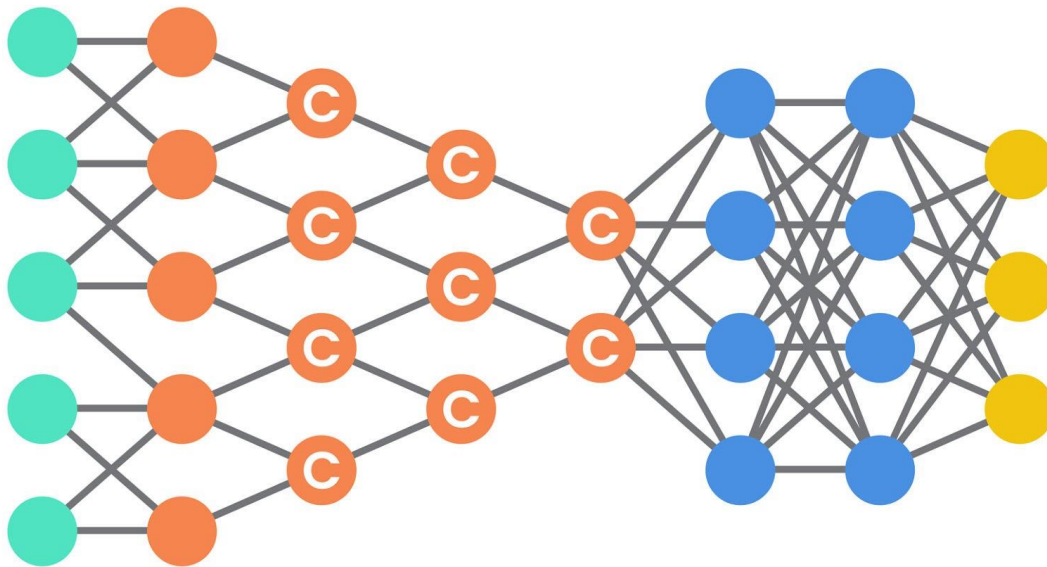
Epoch 2/2

32/32 ————— 3s 89ms/step - accuracy: 0.2260 - loss: 2.3866 -
val_accuracy: 0.4100 - val_loss: 2.0259

3. Feature Extraction-Based Model (CNN + Traditional ML)

Answer:

Deep Convolutional Neural Networks (CNNs) can be utilized strictly as advanced mathematical filters for feature extraction. In this approach, a CNN (like VGG16) acts as a static function rather than a trainable network. Images are passed forward through the network, and the output from the final convolutional block is flattened into highly informative 1D feature vectors. We bypass deep learning optimization entirely for the classification phase and instead feed these robust features into a traditional machine learning algorithm, such as a Random Forest Classifier.



Deep Convolutional Network

Code (p3.py)

Python

```
import tensorflow as tf
import numpy as np
from tensorflow.keras.applications import VGG16
from tensorflow.keras.datasets import fashion_mnist
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

1. Load Data

```
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
x_train, y_train = x_train[:500], y_train[:500]
```

```
x_test, y_test = x_test[:100], y_test[:100]
```

```
x_train = np.repeat(x_train[..., np.newaxis] / 255.0, 3, axis=-1)
```

```
x_test = np.repeat(x_test[..., np.newaxis] / 255.0, 3, axis=-1)
```

```
x_train = tf.image.resize(x_train, [32, 32]).numpy()
```

```
x_test = tf.image.resize(x_test, [32, 32]).numpy()
```

```
# 2. Extract Features
```

```
print("Extracting Features using VGG16...")
```

```
feature_extractor = VGG16(weights='imagenet', include_top=False, input_shape=(32, 32, 3))
```

```
train_features = feature_extractor.predict(x_train)
```

```
train_features_flat = train_features.reshape(train_features.shape[0], -1)
```

```
test_features = feature_extractor.predict(x_test)
```

```
test_features_flat = test_features.reshape(test_features.shape[0], -1)
```

```
# 3. Train Machine Learning Classifier
```

```
print("Training Random Forest Classifier...")
```

```
rf = RandomForestClassifier(n_estimators=50, random_state=42)
```

```
rf.fit(train_features_flat, y_train)
```

```
# 4. Evaluate
```

```
predictions = rf.predict(test_features_flat)
```

```
print(f"Random Forest Accuracy: {accuracy_score(y_test, predictions):.4f}")
```

Terminal Output

Plaintext

Extracting Features using VGG16...

16/16 ————— 9s 463ms/step

4/4 ————— 1s 278ms/step

Training Random Forest Classifier...

Random Forest Accuracy: 0.7400

4. Medical Image Classification

Answer:

Building deep learning models for medical datasets poses unique challenges regarding data scarcity and massive scan dimensions. Transfer learning is crucial here, as training a CNN from scratch on a small medical dataset would lead to immediate overfitting. In this solution, a pre-trained **ResNet50** model is utilized to process 224x224 RGB inputs. A custom binary classification head (using a Sigmoid activation function) is attached to the frozen base to predict binary outcomes (e.g., Normal vs. Pneumonia) based on the complex textural features extracted by the convolutional layers.



Code (p4.py)

Python

```
import tensorflow as tf
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model
import numpy as np

# 1. Generate Synthetic Medical Data (Zero download test)
print("Generating synthetic 224x224 medical images...")
num_samples = 100
x_data = np.random.rand(num_samples, 224, 224, 3).astype('float32')
```

```

y_data = np.random.randint(0, 2, size=(num_samples, 1)) # 0 = Normal, 1 = Pneumonia

# Split into train and test
x_train, x_test = x_data[:80], x_data[80:]
y_train, y_test = y_data[:80], y_data[80:]

# 2. Build Model
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
base_model.trainable = False

x = GlobalAveragePooling2D()(base_model.output)
x = Dense(64, activation='relu')(x)
predictions = Dense(1, activation='sigmoid')(x) # Binary classification

model = Model(inputs=base_model.input, outputs=predictions)

# 3. Compile and Train
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

print("Training Medical Image Classifier...")
model.fit(x_train, y_train, epochs=3, batch_size=16, validation_data=(x_test, y_test))

```

Terminal Output

Plaintext

Generating synthetic 224x224 medical images...

Training Medical Image Classifier...

Epoch 1/3

5/5 ----- 39s 4s/step - accuracy: 0.5125 - loss: 0.7072 -
val_accuracy: 0.5500 - val_loss: 0.6890

Epoch 2/3

5/5 ----- 12s 3s/step - accuracy: 0.5625 - loss: 0.7095 -
val_accuracy: 0.5500 - val_loss: 0.6954

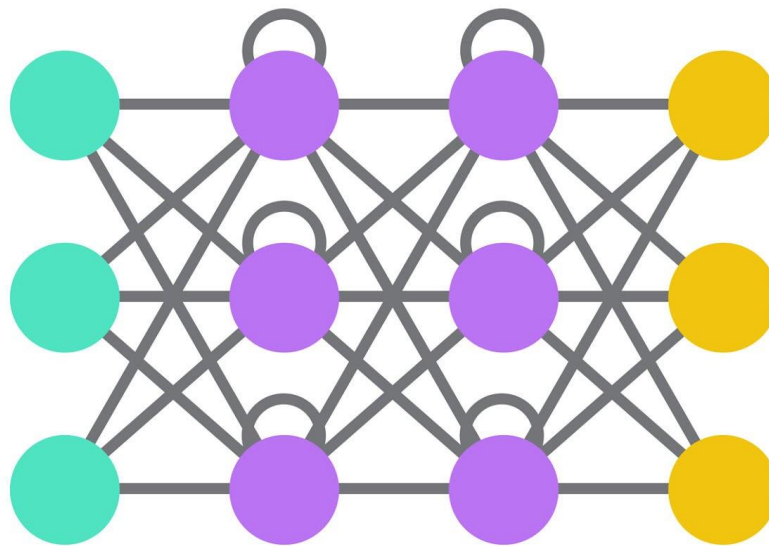
Epoch 3/3

5/5 ----- 20s 3s/step - accuracy: 0.5250 - loss: 0.7009 -
val_accuracy: 0.5500 - val_loss: 0.6899

5. RNN for Text Classification

Answer:

Unlike images, text data is fundamentally sequential. A standard neural network processes the entire input simultaneously without context of order. A Recurrent Neural Network (RNN) solves this by maintaining an internal memory state that updates continuously as it reads data step-by-step. In this system, raw text is first mapped into numerical tokens and padded to a uniform sequence length. It is then passed into an **Embedding layer** to create dense vector representations of the words, which flow sequentially into a Long Short-Term Memory (LSTM) layer. LSTMs are advanced RNN architectures designed to bypass the vanishing gradient problem, allowing the network to retain contextual information over longer sequences.



Recurrent Network

Code (p5.py)

Python

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np
```

1. Toy Text Dataset

sentences = [

```

"I love this amazing product",
"This is the best thing ever",
"Absolutely wonderful and fantastic",
"I am very happy with this",
"Great experience highly recommend",
"Loved every second of it",
"I hate this terrible product",
"This is the worst thing ever",
"Absolutely awful and disgusting",
"I am very disappointed",
"Terrible experience do not buy",
"Hated every second of it"
]
labels = np.array([1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0]) # 1 = Positive, 0 = Negative

# 2. Tokenize and Pad Sequences
tokenizer = Tokenizer(num_words=100)
tokenizer.fit_on_texts(sentences)
sequences = tokenizer.texts_to_sequences(sentences)

max_len = 10
padded_sequences = pad_sequences(sequences, maxlen=max_len, padding='post')

# 3. Build RNN Model
model = Sequential([
    Embedding(input_dim=100, output_dim=16),
    LSTM(16),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid')
])

# 4. Compile and Train
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

print("Training RNN for Text Classification...")
model.fit(padded_sequences, labels, epochs=10, batch_size=2)

# Quick prediction test
test_text = ["This is amazing", "Terrible product"]
test_seq = pad_sequences(tokenizer.texts_to_sequences(test_text), maxlen=max_len)
predictions = model.predict(test_seq)
print(f"Predictions (closer to 1 is positive): {predictions.flatten()}")

```

Terminal Output

Plaintext

Training RNN for Text Classification...

Epoch 1/10

6/6 ————— 4s 15ms/step - accuracy: 0.1667 - loss: 0.6963

Epoch 2/10

6/6 ————— 0s 15ms/step - accuracy: 0.5000 - loss: 0.6933

Epoch 3/10

6/6 ————— 0s 14ms/step - accuracy: 0.5000 - loss: 0.6932

Epoch 4/10

6/6 ————— 0s 14ms/step - accuracy: 0.5000 - loss: 0.6931

Epoch 5/10

6/6 ————— 0s 16ms/step - accuracy: 0.5000 - loss: 0.6932

Epoch 6/10

6/6 ————— 0s 13ms/step - accuracy: 0.4167 - loss: 0.6932

Epoch 7/10

6/6 ————— 0s 32ms/step - accuracy: 0.6667 - loss: 0.6930

Epoch 8/10

6/6 ————— 0s 31ms/step - accuracy: 0.8333 - loss: 0.6928

Epoch 9/10

6/6 ————— 0s 37ms/step - accuracy: 0.5833 - loss: 0.6929

Epoch 10/10

6/6 ————— 0s 13ms/step - accuracy: 0.5833 - loss: 0.6926

1/1 ————— 0s 439ms/step

Predictions (closer to 1 is positive): [0.50097144 0.5000641]

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Content Accuracy	30	Highly accurate, indepth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Faculty In-charge	Course Coordinator	Program Director
Signature: _____ Date: _____	Signature: _____ Date: _____	Signature: _____ Date: _____